

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 05 -

DOCKER COMPOSE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EXEMPLO

O QUE VEM POR AÍ?

Pronto e pronta para elevar as suas habilidades com o Docker Compose?

Se você já brincou com Docker e containers em produção, provavelmente já sentiu que organizar um ou dois containers é fácil, mas quando as coisas começam a crescer, fica complicado manter tudo funcionando numa boa. É aí que entra o Docker Compose. Já imaginou como grandes empresas gerenciam dezenas ou até centenas de containers de maneira organizada e sem dor de cabeça?

Nesta aula, você vai aprender exatamente isso: como orquestrar e gerenciar múltiplos containers com facilidade e eficiência usando apenas um arquivo YAML. Vamos explorar desde o básico até técnicas mais avançadas, como configuração de redes personalizadas, escalabilidade e boas práticas para deixar tudo redondinho e em produção.

HANDS ON

Agora que você já entendeu como funciona o Docker Compose, chegou a hora de colocar a “mão na massa”. Ao longo dos próximos vídeos, vamos sair da teoria e partir para a prática, montando e organizando aplicações reais com múltiplos containers, tudo rodando juntos e sem complicações.

No primeiro vídeo, vamos começar criando nosso primeiro arquivo docker-compose.yml, entendendo a sua estrutura e subindo uma aplicação simples, mas bem funcional. Você vai ver na prática como é fácil conectar containers e deixar tudo rodando de primeira.

Já no segundo vídeo, vamos aprofundar um pouco mais, explorando comandos essenciais que vão facilitar muito seu dia a dia, como docker-compose up, down, ps e logs. Além disso, veremos como usar variáveis de ambiente, portas e volumes para personalizar o ambiente e garantir que os dados fiquem seguros e persistentes.

E, no último vídeo da aula, vamos entrar de vez no mundo avançado do Docker Compose. Você vai aprender técnicas essenciais para escalar serviços automaticamente, definir redes customizadas para uma comunicação segura e ainda gerenciar múltiplos ambientes—seja de desenvolvimento, testes ou produção. Tudo isso sempre seguindo as melhores práticas do mercado.

SAIBA MAIS

Overriding e extensões com múltiplos arquivos Compose

Um dos recursos mais poderosos — e frequentemente subutilizados — do Docker Compose é a capacidade de combinar múltiplos arquivos YAML para representar diferentes ambientes. Isso permite manter um `docker-compose.yml` principal com a configuração base da aplicação e usar arquivos complementares, como `docker-compose.override.yml`, para adaptar detalhes conforme o contexto: **desenvolvimento**, **teste**, **produção**, ou até mesmo **execução local** com recursos adicionais.

Por padrão, quando você executa `dockercomposeup`, o Compose automaticamente carrega o `docker-compose.yml` e, se existir, também o `docker-compose.override.yml`. Esse segundo arquivo "sobrepõe" configurações do primeiro: adiciona, modifica ou remove partes do YAML, desde que os serviços mantenham o mesmo nome.

Mas, você também pode explicitar isso usando a flag `-f` para apontar múltiplos arquivos. Isso dá ainda mais controle sobre o comportamento dos ambientes:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up
```

Comando de prompt 1 – Flag `-f`
Fonte: elaborado pelo autor (2025)

No exemplo acima, o Compose carrega o `docker-compose.yml` com as definições padrão e depois aplica as alterações definidas no `docker-compose.prod.yml`. Essa composição de arquivos permite, por exemplo, trocar a imagem de um container, definir volumes específicos, usar variáveis de ambiente diferentes ou configurar uma rede exclusiva para o ambiente de produção — sem duplicar todo o YAML base.

Essa abordagem modulariza a orquestração. Em vez de manter um arquivo gigante cheio de ifs e comentários indicando "`#produção`" ou "`#apenas dev`", você separa essas lógicas e ainda ganha rastreabilidade (cada ambiente pode ser versionado separadamente).

Builds personalizados no Docker Compose

Nem sempre a imagem que você precisa já está pronta em algum registry público. Muitas vezes, é necessário construir a imagem localmente, a partir de um **Dockerfile** personalizado — e o Docker Compose dá bastante controle sobre esse processo.

Em vez de usar a diretiva **image:**, você pode definir um bloco **build:** para indicar que aquele serviço deve ser construído antes de subir o container. E é aqui que entram três componentes essenciais: **context**, **dockerfile** e **args**.

O **context** define qual pasta será usada como base para o build. Em geral, é o diretório onde estão os arquivos da aplicação e o próprio Dockerfile. Mas você pode apontar para qualquer lugar no seu projeto, o que é útil quando você quer manter os Dockerfiles separados em pastas específicas ou construir imagens a partir de estruturas reutilizáveis.

Já o **dockerfile** permite indicar exatamente qual arquivo deve ser usado para construir a imagem. Isso é útil, por exemplo, quando você tem variações para ambientes diferentes (como **Dockerfile.dev** e **Dockerfile.prod**) e quer alternar entre eles dependendo da situação. Não precisa renomear nada manualmente — só indicar o caminho certo no Compose.

Por fim, os **args** entram em cena quando você quer tornar o build mais dinâmico. Eles funcionam como variáveis que podem ser passadas durante a construção da imagem, e são especialmente úteis para configurar detalhes como o ambiente (**ENV=prod**), a porta padrão, a versão de uma dependência ou qualquer outro parâmetro que você queira controlar sem alterar diretamente o Dockerfile.

```
dev:
  build:
    context: ./frontend
    dockerfile: Dockerfile.dev
  args:
    REACT_APP_API_URL: "http://localhost:3000"
```

Comando de prompt 2 – args
Fonte: elaborado pelo autor (2025)

Com isso, você isola sua lógica de build por ambiente, ganha flexibilidade para fazer ajustes rápidos e ainda evita aquele acúmulo de Dockerfiles quase

idênticos. Além disso, seu time pode clonar o repositório e rodar `dockercomposeup` com builds totalmente adaptados ao seu fluxo de trabalho — sem surpresas.

Se você ainda está usando apenas `image`: em todos os serviços, talvez esteja deixando uma camada poderosa do Compose de lado. Teste usar o `build`: num dos seus serviços e comece a modularizar seus builds... faz a diferença!

Gerenciamento de segredos no Docker Compose

Nem toda variável de ambiente deve ser tratada da mesma forma. Quando falamos de senhas, tokens de acesso ou qualquer informação sensível, é essencial entender como o Docker Compose lida com segredos e variáveis de ambiente — e por que isso importa na prática.

De forma simples, temos dois jeitos principais de passar dados de configuração para nossos containers com Compose: usando `env_file` ou usando a diretiva `secrets`:

O `env_file` é o mais comum e prático. Ele permite carregar variáveis definidas em um arquivo `.env`, como:

```
DB_USER=admin
DB_PASSWORD=supersegredo123
```

Comando de prompt 3 – Carregando variáveis definidas em um arquivo `.env`
 Fonte: elaborado pelo autor (2025)

Depois, no seu `docker-compose.yml`, você pode referenciar esse arquivo:

```
services:
  app:
    image: minha-api
    env_file:
      - .env
```

Comando de prompt 4 – Referenciando arquivo no `docker-compose.yml`
 Fonte: elaborado pelo autor (2025)

Isso é ótimo para valores que você precisa versionar ou compartilhar em ambientes de desenvolvimento. Mas há um detalhe: essas variáveis podem acabar visíveis com comandos como `dockerinspect` ou até nos logs de build, o que não é nada legal quando estamos lidando com dados sensíveis.

É aí que entra a diretiva `secrets`:. Ela oferece uma forma mais segura de lidar com informações confidenciais, especialmente em ambientes de produção. O

conteúdo do segredo fica armazenado em arquivos protegidos e pode ser montado diretamente dentro do container, em um path específico. Isso impede que o valor fique visível no histórico de comandos ou no ambiente como um todo:

```
secrets:
  db_password:
    file: ./secrets/db_password.txt

services:
  db:
    image: postgres
    secrets:
      - db_password
```

Comando de prompt 5 – Diretiva secrets:
Fonte: elaborado pelo autor (2025)

Dentro do container, o conteúdo do segredo estará acessível em `/run/secrets/db_password`, o que é muito mais seguro do que definir `DB_PASSWORD` diretamente no ambiente.

Então, qual desses usar? Em desenvolvimento local, o `env_file` ainda é o “rei” — prático, direto e compatível com praticamente tudo. Mas quando a aplicação começa a ficar séria, com ambiente de staging ou produção, vale a pena migrar os dados mais críticos para `secrets`.

No fim do dia, não é apenas sobre rodar um container — é sobre garantir que o que roda esteja bem protegido. Use `env_file` para o que pode ser compartilhado e `secrets` para aquilo que, se vazar, vai te dar dor de cabeça. **Uma boa stack também é uma stack segura.**

Estratégias de restart no Docker Compose

Sabe aquele container que você sobe e ele roda bonitinho, mas aí um serviço depende de outro que ainda nem iniciou? Ou então aquele container que, por algum motivo temporário, falha e você quer que ele volte automaticamente? A diretiva `restart` no Docker Compose é justamente para isso: garantir resiliência e previsibilidade no comportamento dos seus containers, especialmente em ambientes de desenvolvimento e testes mais exigentes.

A ideia é simples, mas poderosa. Existem algumas opções de política de reinício que você pode configurar:

- **no (padrão)**: não reinicia o container automaticamente. Se ele falhar ou ficar parado, ficará assim até você intervir;
- **always**: sempre reinicia o container, independentemente do motivo da parada. Útil pra serviços essenciais que você quer manter vivos;
- **on-failure**: reinicia apenas se o container terminar com erro (exitcode diferente de 0). Você pode limitar o número de tentativas;
- **unless-stopped**: igual ao **always**, mas se você parar o container manualmente, ele não volta mais sozinho.

Na prática, você pode configurar desta maneira:

```
services:  
  web:  
    image: minha-app  
    restart: always
```

Comando de prompt 6 – Opções de política de reinício
Fonte: elaborado pelo autor (2025)

Mas não é colocar **restart: always** em tudo. O segredo é entender o papel de cada serviço no seu projeto. Por exemplo:

- Um banco de dados? **restart: unless-stopped** costuma ser uma boa ideia. Você quer que ele sempre volte, mas respeite sua decisão caso precise desligar;
- Uma API que depende do banco? Pode ser **on-failure** para evitar loops infinitos caso o erro seja de configuração;
- Um serviço que processa fila? Talvez **restart: always** faça sentido pra garantir que ele esteja sempre ativo.

E claro, tudo isso ajuda muito quando você usa Compose em conjunto com CI/CD ou ambientes locais com múltiplos containers. Já pensou rodar **dockercomposeup** e metade dos serviços quebrarem por conta de ordem de inicialização? Com as estratégias de restart bem definidas, seu ambiente fica mais robusto — e você perde menos tempo “debugando” coisas que poderiam estar cobertas desde o início.

Então, da próxima vez que configurar seu Compose, pense não só em "fazer rodar", mas em "fazer rodar direito, mesmo quando der errado". É aí que o **restart** brilha.

Integração com CI/CD usando Docker Compose

Uma das maiores vantagens do Docker Compose é a facilidade que ele traz para reproduzir ambientes complexos com um simples comando. Mas essa simplicidade não serve apenas para desenvolvedores rodando apps localmente — ela é perfeita para pipelines de CI/CD que precisam subir, testar e destruir ambientes com rapidez e precisão.

Vamos pensar no seguinte: se você já tem um **docker-compose.yml** que roda sua aplicação localmente, por que não usar esse mesmo arquivo para rodar seus testes automatizados num ambiente isolado, dentro de uma pipeline? A ideia é justamente essa: trazer consistência entre o que roda na máquina do dev e o que roda na sua esteira de build.

Imagine um workflow de CI onde você tem um app Node.js e um banco Postgre. Em vez de depender de serviços externos ou mockar tudo, você usa o Compose pra subir os dois containers, roda os testes de integração e depois derruba tudo. Tudo controlado pelo **docker-compose up** e **down** dentro do seu script de pipeline.

Outro ponto importante é simular a produção. Usando Compose com diferentes arquivos (como **docker-compose.override.yml** ou **docker-compose.prod.yml**), você consegue aproximar seu ambiente de CI do que realmente roda em produção. Isso evita surpresas desagradáveis quando você faz o deploy.

E tem mais: o Compose pode ser usado para otimizar o tempo de build. Ao definir volumes e caching corretamente, você consegue acelerar testes e builds entre diferentes etapas da pipeline. Sem contar que o Compose é suportado por praticamente todos os provedores de CI modernos, seja GitHub Actions, GitLab CI, Bitbucket ou Jenkins.

No fim, é sobre isso: se você já montou sua aplicação como um conjunto de serviços com Compose, aproveite esse mesmo setup na sua esteira. Além de evitar inconsistências, você ganha agilidade, previsibilidade e menos “dor de cabeça”

quando o ambiente de testes começa a se comportar como o de produção. Ninguém quer ouvir aquele famoso "na minha máquina funciona" em pleno 2025, não é?!

EMSE

MERCADO, CASES E TENDÊNCIAS

Livro: A Developer's Essential Guide to Docker Compose

Autor: Emmanouil Gkatzouras

Editora: Packt Publishing

Ano: 2022

Para aprofundar ainda mais o seu domínio sobre Docker Compose, abordando desde fundamentos até estratégias práticas para desenvolvimento diário, CI/CD e deploy em produção, confira especialmente os capítulos sobre **“Comunicação entre Microservices”** e **“Como traduzir arquivos Compose em implantações no Kubernetes”**.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a orquestrar múltiplos containers com o Docker Compose. Começamos entendendo o que é e como surgiu essa ferramenta, exploramos a estrutura do arquivo docker-compose.yml, vimos exemplos práticos de aplicações multicontainer e comandos essenciais do dia a dia.

Encerramos com técnicas mais avançadas como escalonamento de serviços, redes customizadas e estratégias para organizar ambientes de desenvolvimento e produção. Agora você tem o que precisa para estruturar projetos mais robustos e profissionais com muito mais facilidade.

REFERÊNCIAS

DOCKER Compose overview. **Docker Documentation**. 2024. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 20 ago. 2025.

DOCKER Compose: **The Complete Guide to Docker Compose**. 2020. Disponível em: <https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-compose-on-ubuntu-20-04>. Acesso em: 20 ago. 2025.

GKATZIOURAS, E. **A Developer's Essential Guide to Docker Compose**. Birmingham: Packt Publishing, 2022.

PALAVRAS-CHAVE

DevOps. Integração Contínua. Orquestração de Containers.

EMSE



POSTECH